EXCEPCIONES EN JAVA

Ingeniería de Sistemas / Ingeniería de software | Docente: John Carlos Arrieta Arrieta

Fundamentos, buenas prácticas y retos de programación

Clase práctica con ejemplos y retos

Objetivos de aprendizaje

Al finalizar, podremos:

- Diferenciar errores de compilación, ejecución y lógica.
- Interpretar la jerarquía Throwable.
- Usar try, catch, finally, throw y throws.
- Distinguir checked y unchecked exceptions.
- Diseñar excepciones personalizadas y resolver retos.

Agenda · 150 minutos

0–15 min Contexto, errores e historia

15–40 min Excepciones y sintaxis esencial

40–70 min Ejemplos guiados y buenas prácticas

70–95 min Checked, unchecked y jerarquía

95–115 min Excepciones personalizadas

115–145 min Retos por parejas

145–150 min Cierre y verificación

¿Qué ocurre cuando algo falla?

Una aplicación intenta leer un archivo inexistente, dividir entre cero o convertir “abc” a número.

Pregunta inicial: ¿debe terminar todo el programa o podemos responder de forma controlada?

Idea clave: separar el flujo normal de la gestión de situaciones excepcionales.

Tipos de errores

Compilación: sintaxis o tipos incorrectos; el programa no compila.

Ejecución: problema mientras corre; puede producir una excepción.

Lógica: compila y corre, pero entrega un resultado incorrecto.

Error de infraestructura: disco, memoria, red; su tratamiento depende del caso.

Tres ejemplos de fallos

```
// Compilación: tipo incompatible
// int edad = "veinte";

// Ejecución: ArithmeticException
int resultado = 10 / 0;

// Lógica: cálculo incorrecto, sin excepción
int promedio = (10 + 20) / 3; // ¿Era entre 2?
```

Antes de las excepciones

Enfoques históricos frecuentes:

- Códigos de retorno: -1, 0, 1...
- Variables globales de error (p. ej., errno en C).
- Comprobaciones manuales después de cada llamada.

Limitaciones: se ignoran resultados, se mezcla la lógica de negocio con la de errores y se pierde contexto.

Código de retorno vs. excepción

```
// Estilo basado en código de retorno
int estado = guardarArchivo();
if (estado != 0) { /* investigar el fallo */ }

// Estilo basado en excepciones
try {
    guardarArchivo();
} catch (IOException e) {
    System.err.println(e.getMessage());
}
```

¿Qué es una excepción?

Objeto que representa una condición anómala durante la ejecución.

Cuando se lanza, Java interrumpe el flujo normal y busca un manejador compatible.

Si no encuentra uno, la excepción se propaga hasta terminar el hilo correspondiente.

No todo error del programa se representa mediante una excepción.

Anatomía de try / catch / finally

```
try {
    // Operación que puede fallar
} catch (TipoExcepcion e) {
    // Tratamiento específico
} catch (OtraExcepcion e) {
    // Otro tratamiento, del más específico al general
} finally {
    // Limpieza; normalmente se ejecuta siempre
}
```

try y catch: conversión de datos

```
String entrada = "abc";
try {
    int numero = Integer.parseInt(entrada);
    System.out.println(numero);
} catch (NumberFormatException e) {
    System.out.println("Ingrese un entero válido");
}
System.out.println("El programa continúa");
```

finally: liberar recursos

finally se usa para limpieza, pero no garantiza ejecutarse si la JVM termina abruptamente.

En recursos AutoCloseable, preferimos try-with-resources.

Evitar return en finally: puede ocultar fallos.

```
try {
    System.out.println("Procesando...");
} finally {
    System.out.println("Fin del intento");
}
```

try-with-resources

```
import java.nio.file.*;
import java.io.*;


try (BufferedReader lector =
         Files.newBufferedReader(Path.of("datos.txt"))) {
    System.out.println(lector.readLine());
} catch (IOException e) {
    System.err.println("No se pudo leer: " + e.getMessage());
}
// lector.close() se ejecuta automáticamente
```

throw: lanzar una excepción

```
static void registrarEdad(int edad) {
    if (edad < 0) {
        throw new IllegalArgumentException(
            "La edad no puede ser negativa");
    }
    System.out.println("Edad registrada: " + edad);
}
```

throws: declarar propagación

```
import java.io.IOException;
import java.nio.file.*;

static String leerConfig() throws IOException {
    return Files.readString(Path.of("config.txt"));
}

// El llamador debe manejar o declarar IOException.
// throws NO captura la excepción.
```

Propagación y pila de llamadas

main() → procesar() → leerArchivo()

Si leerArchivo() lanza IOException y no la captura, se propaga hacia procesar() y después hacia main().

El stack trace muestra la ruta de llamadas.

Regla: capturar donde realmente podamos recuperar, traducir o informar adecuadamente.

Ejemplo práctico · retiro bancario

Reglas: saldo inicial $100.000; retirar un valor positivo que no supere el saldo.

¿Qué falla? Monto negativo, fondos insuficientes, entrada inválida.

Separaremos validación del dominio y manejo en el punto de entrada.

```
static int retirar(int saldo, int monto) {
    if (monto <= 0)
        throw new IllegalArgumentException("Monto inválido");
    if (monto > saldo)
        throw new IllegalStateException("Fondos insuficientes");
    return saldo - monto;
}
```

Ejemplo práctico · ejecución

```
int saldo = 100_000;
try {
    saldo = retirar(saldo, 120_000);
    System.out.println("Saldo: " + saldo);
} catch (IllegalArgumentException e) {
    System.out.println("Validación: " + e.getMessage());
} catch (IllegalStateException e) {
    System.out.println("Operación rechazada: " + e.getMessage());
}
// Pregunta: ¿cuánto vale saldo después del fallo?
```

Checked vs. unchecked

Checked (verificadas): subclases de Exception, excepto RuntimeException. El compilador obliga a capturarlas o declararlas. Ej.: IOException.

Unchecked (no verificadas): RuntimeException y Error y sus descendientes. No requieren catch ni throws. Ej.: NumberFormatException.

«Declaradas/no declaradas» puede ser ambiguo: una unchecked también puede incluirse en throws.

Ejemplo checked: IOException

```
static String cargar() throws IOException {
    return Files.readString(Path.of("clientes.csv"));
}

public static void main(String[] args) {
    try {
        System.out.println(cargar());
    } catch (IOException e) {
        System.err.println("Archivo no disponible");
    }
}
```

Ejemplo unchecked: NumberFormatException

```
static int convertir(String texto) {
    return Integer.parseInt(texto);
}

public static void main(String[] args) {
    try {
        System.out.println(convertir("12x"));
    } catch (NumberFormatException e) {
        System.out.println("Formato numérico inválido");
    }
}
```

Jerarquía de Throwable

java.lang.Object
└─ java.lang.Throwable
  ├─ java.lang.Error (unchecked)
  │ └─ OutOfMemoryError
  └─ java.lang.Exception
    ├─ IOException (checked)
    └─ RuntimeException (unchecked)
      ├─ IllegalArgumentException
      ├─ NullPointerException
      └─ IllegalStateException

Orden de catch y multicatch

```
try {
    ejecutar();
} catch (NumberFormatException e) {
    System.out.println("Número inválido");
} catch (RuntimeException e) {
    System.out.println("Fallo de ejecución");
}

// Multicatch para tipos no relacionados:
catch (IOException | SecurityException e) {
    System.err.println(e.getMessage());
}
// El bloque multicatch es un ejemplo independiente.
```

Excepciones personalizadas

Útiles cuando un fallo tiene significado de negocio.

Extender RuntimeException para unchecked o Exception para checked.

Usar nombres claros, mensaje útil y, cuando corresponda, preservar la causa.

```
public class SaldoInsuficienteException
        extends RuntimeException {
    public SaldoInsuficienteException(String mensaje) {
        super(mensaje);
    }
}
```

Aplicación de la excepción personalizada

```
static int retirar(int saldo, int monto) {
    if (monto <= 0)
        throw new IllegalArgumentException("Monto inválido");
    if (monto > saldo)
        throw new SaldoInsuficienteException(
            "Disponible: " + saldo + ", solicitado: " + monto);
    return saldo - monto;
}
// SaldoInsuficienteException se define en otra clase.
```

Personalizada checked: alternativa

```
public class LimiteDiarioException extends Exception {
    public LimiteDiarioException(String mensaje) {
        super(mensaje);
    }
}

static void validarLimite(int monto)
        throws LimiteDiarioException {
    if (monto > 500_000)
        throw new LimiteDiarioException("Supera límite diario");
}
```

Buenas prácticas

- Capturar excepciones específicas y no silenciarlas.
- No usar excepciones para controlar el flujo normal.
- Registrar contexto sin exponer secretos.
- No capturar Error salvo escenarios muy especiales.
- Preservar la causa: new MiExcepcion("mensaje", e).
- Preferir try-with-resources para recursos cerrables.

Reto 1 · Conversor robusto

Tiempo: 8 minutos | Individual

Crear convertirEdad(String texto).

Requisitos: convertir a int, rechazar negativos y mostrar mensajes distintos para formato inválido y edad negativa.

Casos: "25", "abc", "-3".

Extra: ¿dónde capturarías cada excepción?

Reto 2 · Lectura segura

Tiempo: 10 minutos | Parejas

Leer un archivo de texto usando Files.readString().

Gestionar IOException con un mensaje comprensible.

Mostrar qué cambia si el método declara throws y el llamador captura.

Extra: probar con archivo inexistente.

Reto 3 · Cajero automático

Tiempo: 12 minutos | Parejas

Crear Cuenta con saldo y retirar(monto).

Crear SaldoInsuficienteException personalizada.

Validar monto positivo y saldo disponible.

Probar 3 casos: retiro válido, negativo, superior al saldo.

Extra: añadir límite diario checked.

Cierre · comprobación rápida

1. ¿En qué se diferencian throw y throws?
2. ¿Cuándo el compilador obliga a tratar una excepción?
3. ¿Por qué conviene try-with-resources?
4. ¿Qué pasa si no hay catch compatible?
5. ¿Cuándo crearías una excepción de dominio?

Salida: explicar un caso y justificar cómo manejarlo.

Referencias para continuar

Oracle Java Tutorials: Exceptions (conceptos fundamentales).

Java Language Specification, capítulos 11 y 14.

Java SE API: Throwable, Exception, RuntimeException, IOException.

Documentación: docs.oracle.com/javase/tutorial/essential/exceptions/

Especificación: docs.oracle.com/javase/specs/